Strata — Hướng dẫn Child

Dành cho người biên soạn tài liệu

strata-kb v1.34.5 · 2026-11-05

1. Tài liệu này dành cho ai

Bạn biên soạn tài liệu. Bạn đưa một PDF nguồn vào, kiểm tra xem bộ tách đã làm gì với nó, cho nó được tóm tắt, kiểm tra hợp lệ, ký duyệt rồi publish lên hub, nơi mọi người khác có thể tìm kiếm. Một repository làm việc này là **child**.

> **Không có gì bạn biên soạn tìm kiếm được cho đến khi một pull request trên hub được merge.** `.kb/` cục bộ chỉ là bàn nháp. Hub mới là thế giới đã publish.

Repo child có làm	Repo child không làm
Ingest tài liệu nguồn và giữ bốn lớp (L0 đến L3)	Chạy dịch vụ tìm kiếm toàn công ty, MCP server hay web UI: việc đó của hub
Tóm tắt, kiểm tra hợp lệ, duyệt và publish chúng	Tự duyệt bất cứ thứ gì: duyệt là hành động của con người và không bao giờ tự động
Đọc tri thức đã publish trên hub (`kb query`, MCP)	Publish ticket hay tri thức về code: repo `ba` và `dev` có tài liệu riêng

Repo hub có thể biên soạn tài liệu bằng cùng các bước này cho đến khi publish (mục 3), nên tài liệu về hub trỏ về đây thay vì lặp lại và liệt kê các điểm khác.

Các khái niệm dùng chung, như các lớp L0 đến L3 và trích dẫn `kb-context`, được giải thích trong tài liệu kiến trúc.

Slash command. Mọi slash command trong tài liệu này đều có cho Claude Code, GitHub Copilot và Cursor, trừ các ngoại lệ sau:

- `/kb-summarize` không có prompt cho Copilot. Copilot nạp `.github/instructions/kb-summarize.instructions.md` khi bạn làm việc trong `.kb/**`. Cursor còn có thêm một rule, `.cursor/rules/kb-summarize.mdc`.
- `/kb-ingest` và `/kb-publish` trong Claude Code là skill (`.claude/skills/<name>/`), không phải file trong `.claude/commands/`. Bạn vẫn gõ slash command như thường.
- `/kb-l3-fix` chỉ có ích khi tính năng sửa L3 bằng AI đang bật (mục 3.3).
- Repo child không có `/kb-code-sync` và không có `/kb-auto`.

Các lệnh child có: `/kb-init`, `/kb-docker-setup`, `/kb-mcp-setup`, `/kb-ingest`, `/kb-summarize`, `/kb-l3-fix`, `/kb-approve`, `/kb-publish`.

---

2. Thiết lập lần đầu

2.1 Cài đặt và dựng khung

```
pip install "strata-kb[ingest]"
mkdir my-kb && cd my-kb && git init
kb init --kind child
```

Bạn cần Python 3.11 trở lên và Git. Extra `ingest` mang theo pipeline xử lý PDF; không có nó thì mọi lệnh khác vẫn chạy, hợp với máy chỉ truy vấn hoặc publish. Không muốn cài Python cục bộ để ingest? Dùng Docker (mục 2.3).

`kb init --kind child` ghi `.kb/config.yaml`, `.kb/index.yaml`, `source/.gitignore`, `docker-compose.yml`, `.gitattributes`, phần nối MCP (`.mcp.json`, `.cursor/mcp.json`), workflow CI `.github/workflows/kb-publish.yml`, `QUICKSTART.md` và tám skill cùng command cho cả ba trợ lý. Chạy lại là an toàn: nó tạo file còn thiếu và làm mới file dựng khung, nhưng bỏ qua `.kb/config.yaml` và `.kb/index.yaml` trừ khi bạn thêm `--force` (xem phụ lục).

2.2 Trỏ tới hub và đăng ký repo

```
# .kb/config.yaml
kind: child
repo_id: safety
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com   # optional: publish from CI
```

- **`hub`** là URL git hoặc đường dẫn của hub. `kb query`, `kb get` và `kb tags` đọc nó qua một bản clone cache, còn `kb publish` không có `intake:` thì publish vào đó.
- **`intake`** là dịch vụ intake của hub. Đặt nó để publish từ CI của chính bạn (mục 4.2): repo này không giữ bí mật nào.
- **`repo_id`** được điền sẵn từ tên thư mục và trở thành tiền tố của mọi trích dẫn repo này sinh ra (`safety:handbook §4.2`), nên hãy chọn ngắn và bền; đổi sau này sẽ bỏ mồ côi mục cũ trên hub. Nó phải **khớp chính xác** với id mà repo này được đăng ký trong `federation/registry.yaml` của hub. Hãy nhờ người quản trị hub thêm vào. Khi file đó không còn rỗng, hub trở thành *được quản trị (governed)*: `kb publish` qua git từ chối mọi repo-id không có trong đó và từ chối push trực tiếp, nên mọi child đều phải được đăng ký, không riêng những child dùng `intake:`.

Không bao giờ đặt token vào `hub:` hay `intake:`. File này được commit, nên token sẽ nằm vĩnh viễn trong lịch sử. `kb doctor` sẽ cảnh báo; hãy viết lại URL, truyền URL có thông tin xác thực qua biến môi trường `STRATA_KB_HUB` (nó ghi đè file), và đổi token. Mọi máy dùng URL hub có thông tin xác thực cần **Git 2.31 trở lên**: Git cũ hơn bỏ qua cơ chế giữ token ra khỏi config của bản clone, và mọi lệnh hub đều lỗi xác thực. Runner CI thì ổn.

`hub:` và `STRATA_KB_HUB_URL` là hai giá trị khác nhau: `hub:` là URL git hoặc đường dẫn của repo hub; `STRATA_KB_HUB_URL` là URL HTTP gốc của hub, dùng cho MCP.

2.3 File nguồn và image ingest

PDF nguồn thường có bản quyền hoặc mang tính bảo mật. Hãy để chúng trong `source/`, thư mục mà `kb init` đã bảo Git bỏ qua (`sources/` cũng được bỏ qua). **Đừng commit chúng.** Cơ sở tri thức mang phần văn bản đã trích xuất do bạn kiểm soát; bản gốc ở nguyên chỗ hợp pháp của nó. `kb doctor` cảnh báo khi git đang theo dõi một file trong hai thư mục này.

Để ingest mà không cần cài Python cục bộ, hãy kéo image ingest:

```
kb docker-setup        # in your assistant: /kb-docker-setup
```

Nó kiểm tra Docker và kéo image Strata, để ingest chạy trong container (mục 4.3). Thêm `--no-docker` nếu chỉ muốn in ra các lệnh. Bỏ qua nếu bạn đã cài extra `ingest`.

2.4 Kết nối trợ lý của bạn

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) trỏ tới hub qua hai placeholder, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`. Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Nó hỏi URL HTTP gốc và token của hub, ghi cả hai vào `.env`, bảo đảm git bỏ qua file đó, rồi xác minh chúng với hub, nên URL sai và token bị từ chối cho ra hai lỗi khác nhau. Hãy lấy token từ người quản trị hub.

Hãy dùng ô nhập ẩn hoặc biến `STRATA_KB_HTTP_TOKEN` thay cho `--token`. Cờ đó để token lại trong lịch sử shell và trong danh sách tiến trình mà người dùng khác trên máy đọc được. `kb mcp-setup` cũng cảnh báo khi URL là `http://` thuần tới một host không phải loopback, vì lúc đó token đi qua mạng ở dạng rõ.

Cấu hình MCP đọc các biến này từ môi trường của bạn, không đọc từ `.env`, vào lúc trợ lý khởi động. Hãy nạp file (`set -a; source .env; set +a`, hoặc dùng direnv) rồi khởi động lại trợ lý.

Một phiên bản cho mọi repo. Mọi repo chạy cùng phiên bản strata-kb của hub. Hãy nâng cấp hub trước. Sau đó, khi `kb mcp-setup` đã chạy (ở trên), chạy `kb doctor` trong repo này: nó cài phiên bản của hub, đặt các bản ghim `strata-kb==<phiên bản của hub>` trong `.github/workflows/`, và làm mới skill cùng command. Trong CI nó không bao giờ cài, đặt lại pin hay làm mới skill; một job CI chạy `kb doctor` với `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` được đặt sẽ báo phiên bản lệch như một lỗi (không workflow nào được scaffold chạy lệnh này). Khi `STRATA_KB_HUB` hoặc `--hub` được đặt, `kb doctor` bỏ qua đường MCP và bước kiểm tra phiên bản, chỉ báo rằng phiên bản chưa được kiểm tra với hub. Các file dưới `.github/instructions/` và `.cursor/rules/` (các file `kb-summarize` của Copilot và Cursor) chỉ được làm mới qua `kb init`. Bản ghim là có lý do: một `pip install strata-kb` không ghim trong job có `id-token: write` sẽ để mỗi lần chạy lấy bất cứ gì PyPI đang phát, trong một job có quyền ghi vào hub.

2.5 Hỏi xem hub có tự merge repo của bạn không

Admin hub có thể bật **auto-merge** cho từng repo trong trang Registry của hub. Khi đó hub merge PR publish của bạn ngay sau khi intake mở nó, không có review phía hub, và `kb publish` in `kb publish: merged on the hub — <link PR>` thay vì `kb publish: PR on the hub — <link PR>`. Hãy hỏi người quản trị hub bạn thuộc trường hợp nào: khi auto-merge tắt, một người trên hub review và merge mọi PR publish.

---

3. Công việc hằng ngày

Quy trình biên soạn, từng tài liệu một:

Việc	Lệnh	Kết quả
Đưa một PDF vào	`kb ingest source/<file>.pdf --id <id>` hoặc `/kb-ingest`	`.kb/<id>/` có L3 và khung L1/L2 đang chờ; bước kiểm tra L3 chạy
Review L3 bị đánh dấu	`kb l3 report <id>`, rồi `kb l3 confirm <id> --section <s>`	tài liệu được đi tiếp
Sửa cấu trúc L3 bằng AI (không bắt buộc)	`kb l3 fix on`, `/kb-l3-fix <id>`	các bản nháp mà máy kiểm tra đối chiếu với PDF
Tóm tắt	tự động khi ingest, nếu không thì `kb summarize` hoặc `/kb-summarize`	L1/L2 được điền
Kiểm tra hợp lệ	`kb build`	`kb build: OK`
Ký duyệt	commit, rồi `kb approve <id>` hoặc `/kb-approve`	các section thành `reviewed`
Publish	`kb publish` hoặc `/kb-publish`	một PR trên hub (hoặc một lần merge, nếu auto-merge bật)

3.1 Ingest một tài liệu

```
kb ingest source/employee-handbook.pdf \
 --id hr-handbook \
 --tags hr,policy,benefits \
 --revision "2026 edition"
```

Trong trợ lý, `/kb-ingest <file>` hỏi id, tag và revision trước rồi mới chạy lệnh; nếu có Docker và `docker-compose.yml` thì nó dùng container (mục 4.3).

Cờ	Tác dụng	Lời khuyên
`--id`	Id vĩnh viễn của tài liệu	Ngắn, chữ thường, nối bằng gạch ngang. Nó xuất hiện trong mọi trích dẫn mãi mãi.
`--tags`	Nhãn lọc trước khi tìm	2 đến 5. Hãy nghĩ theo cách người đọc thu hẹp tìm kiếm.
`--revision`	Nhãn phiên bản	Luôn đặt: nó xuất hiện trong mọi trích dẫn.
`--sections`	Chỉ các chương này, vd `5,6`	Chạy thử trên tài liệu dài, hoặc ingest lại một chương.
`--chapter-pattern`, `--appendix-pattern`, `--attachment-pattern`	Regex tiêu đề	Mặc định là `Chapter N`, `Appendix X`, `Attachment N`; giá trị bạn dùng lần trước được nhớ.
`--no-bookmarks`	Tách theo mẫu tiêu đề, không theo outline của PDF	Chỉ khi outline thiếu hoặc sai.
`--no-summarize`	Bỏ qua bước tóm tắt tự động	Bạn muốn xem kết quả tách trước.
`--llm claude|copilot|none`	Chọn runner tóm tắt	Mặc định: tự phát hiện.
`--no-llm-review`	Bỏ qua nhận định của agent về các phát hiện L3	Mục 3.2.

Ingest ghi ra:

```
.kb/hr-handbook/
├── _manifest.yaml              L1: every section, all "pending"
├── ch4-benefits.md             L2: tables in place, prose blank
├── ch4-benefits.raw.md         L3: the full original
├── _l3check.yaml               the L3 check (section 3.2)
└── assets/<sha256>.webp        figures, stored once each
```

Nó còn giữ một cache tạm trong `.kb-work/`; hãy giữ thư mục này ngoài git. Lần chạy đầu trên một máy sẽ tải mô hình bố cục trang (khoảng 500 MB) rồi cache lại.

Hãy đọc báo cáo ingest. Nó nêu tên từng quyết định của bộ tách: tiêu đề bị hạ xuống thành văn bản, id phải tự sinh, tiêu đề đánh số sai thứ tự, id trùng bị đổi tên, bookmark không khớp được. Nhiều id dự phòng hoặc tiêu đề sai

thứ tự thường nghĩa là outline sai: hãy thử `--no-bookmarks` kèm một mẫu tiêu đề.

Ingest lại. Không có `--sections`, ingest là thay thế toàn bộ: mọi file Markdown của tài liệu bị xoá và ghi lại, kể cả những bản tóm tắt bạn đã review. Với `--sections 6`, chỉ file và mục manifest của chương đó được ghi lại; phần còn lại, kể cả tóm tắt đã review, không bị đụng tới. Một giá trị `--sections` không khớp tiêu đề nào sẽ **xoá** chương đó, và báo cáo nói rõ điều này: hãy đọc nó.

Ingest một file Markdown. File `.md` hoặc `.markdown` đi thẳng vào L3, không cần extra ingest và không cần PDF:

```
kb ingest source/policy.md --id policy
```

Chỉ các dòng tiêu đề bị ghi lại, theo cùng quy tắc id như PDF (tiêu đề đánh số giữ số của nó, tiêu đề không đánh số nhận id dạng slug dưới mục cha, và lồng theo cấp `#`); mọi byte khác của file vào L3 nguyên vẹn. Ảnh `.png` và `.webp` cục bộ nằm trong thư mục nguồn được chép vào `assets/<sha256>.<ext>` và liên kết được ghi lại; ảnh khác giữ nguyên, kèm cảnh báo. Ingest từ chối file có khối code rào (fence) hoặc khối HTML chứa dòng `##` hoặc `###` (nó liệt kê số dòng: hãy dời dòng ra ngoài hoặc thụt lề), và từ chối id đã dùng cho loại kia: `'<id>' was ingested from a PDF — ingest the Markdown under a new --id`, và ngược lại. `kb l3 fix apply`, `kb l3 lint` và `kb l3 fix pages` nhận `--source <file>` (`--pdf` vẫn dùng được); tài liệu Markdown không có ảnh trang, nên `kb l3 fix pages` và `--from-image` từ chối với `source is Markdown: no page images`. Markdown nguồn là bản gốc: muốn đổi văn bản, hãy sửa nó rồi ingest lại. Việc này đặt lại bản ghi AI-fix, nên hãy chạy `/kb-l3-fix` và xác nhận lại. `kb authored <doc>` vẫn đóng băng tài liệu.

3.2 Kiểm tra L3 với PDF

Sau khi ghi L3, `kb ingest` so từng section với lớp văn bản của chính PDF và ghi các phát hiện vào `.kb/<id>/_l3check.yaml` (hãy commit; file này không bao giờ được gửi lên hub). Một section bị đánh dấu khi id của nó là id dự phòng, sai thứ tự, thiếu, thừa hoặc lỗi chữ, một đoạn bị ngắt giữa chừng, hoặc các trang không có lớp văn bản để kiểm tra. Một agent (CLI `claude` hoặc `copilot` mà nó tìm thấy, với `llm.model` từ `.kb/index.yaml`) thêm nhận định cho mỗi phát hiện về đoạn bị ngắt: `spurious`, `intentional` hoặc `unsure`. Agent không bao giờ gỡ dấu và không bao giờ sửa L3.

Tài liệu bị đánh dấu không đi tiếp được. `kb summarize` bỏ qua nó (exit 1), `kb build` báo lỗi, còn `kb publish` và `kb ci-publish` từ chối. Khi đó ingest in `Summarize skipped until the L3 review is confirmed`. Không có gì bị đánh dấu nghĩa là không có gì phải làm.

```
kb l3 report hr-handbook                        # findings, evidence, verdicts (exit 1 while any)
kb l3 confirm hr-handbook --section 4.12        # a reviewer checked this one
kb l3 confirm hr-handbook --all --yes           # every flagged section
```

Mở trang PDF mà báo cáo nêu và đối chiếu. Chỉ xác nhận những gì bạn đã kiểm tra; `--by 'name <email>'` ghi đè danh tính git. Xác nhận chỉ có hiệu lực khi L3 của section không đổi; ingest lại một section đã đổi sẽ hỏi lại.

3.3 Sửa cấu trúc L3 bằng AI (không bắt buộc)

Ingest có thể để lại lỗi cấu trúc trong L3: id dự phòng, cấp tiêu đề sai, đoạn văn bị ngắt giữa chừng, khối bị lặp, đầu trang hoặc chân trang còn sót trong văn bản. Tính năng sửa L3 bằng AI cho một trợ lý sửa cấu trúc đó, còn máy kiểm tra rằng trợ lý không đổi gì khác. Mặc định tính năng này tắt (`l3_fix.enabled` trong `.kb/config.yaml`), và L3 vẫn là văn bản của PDF.

```
kb l3 fix on                                    # also adds .kb-work/ to .gitignore
kb ingest source/handbook.pdf --id hr-handbook # records a baseline; does not summarize
kb l3 lint hr-handbook --pdf source/handbook.pdf
# run /kb-l3-fix hr-handbook in your assistant: it writes drafts under .kb-work/
kb l3 fix apply hr-handbook --pdf source/handbook.pdf --assistant claude-code
kb l3 report hr-handbook                        # AI fixes first, each with a git diff hint
```

```
kb l3 confirm hr-handbook --section 4.12       # a human checked it
kb summarize
```

- `--assistant` là `claude-code`, `copilot`, `cursor` hoặc `other`: công cụ đã viết các bản nháp. `kb l3 lint` chỉ mang tính tham khảo (quy tắc bố cục R1 đến R9, exit 1 khi có vấn đề).
- `kb l3 fix apply` kiểm tra từng bản nháp với PDF và không ghi gì cho bản nháp không đạt. Mỗi dòng kết quả có dạng `§<name> applied → <new ids>`, `§<name> unchanged` hoặc `§<name> rejected: <check>: <detail>`; nó exit 1 khi có bất kỳ từ chối nào. Các phép kiểm tra (bảng khớp từng byte, từng từ đối chiếu với lớp văn bản của PDF) được mô tả trong skill.
- Với section bị quét ảnh hoặc font hỏng, `kb l3 fix pages <id> --pdf <pdf> --page N` xuất các trang PNG vào `.kb-work/<id>/pages/` cho trợ lý, và `apply --from-image <name>` bỏ qua bước kiểm tra từ. Với section có lớp văn bản dùng được, nó chỉ được dùng cho các dòng heading (heading bị lỗi font); phần còn lại của draft vẫn được kiểm tra từng từ. Bản sửa đó bị đánh dấu chưa được kiểm chứng và `kb l3 report` xếp nó lên đầu: hãy đọc nó đối chiếu với PDF.
- Khi công tắc bật, ingest không tóm tắt. Khi một bản sửa của AI đang chờ review, hoặc L3 bị sửa ngoài `apply`, cùng cổng chặn như ở 3.2 vẫn giữ. Hãy commit `_l3fix.yaml`; file này không bao giờ được gửi lên hub. `kb l3 fix off` tắt công tắc.

AI không bao giờ xác nhận. Máy kiểm tra bảng và từ, không kiểm tra được cấp tiêu đề hay đoạn văn đã gộp có đúng ý tác giả hay không. Bạn đọc `git diff` đối chiếu với PDF rồi mới xác nhận.

3.4 Tóm tắt

Đây là bước duy nhất mà model viết văn xuôi. `kb ingest` chạy nó tự động khi tìm thấy CLI `claude` hoặc `copilot` (cấu hình: `llm:` trong `.kb/index.yaml`; `--llm` chỉ định một runner). Nếu không, các section ở nguyên trạng thái `pending`:

```
kb status                    # sections still pending summarization
kb summarize hr-handbook     # run or retry
```

Không có LLM CLI thì chạy `/kb-summarize` trong trợ lý: nó đọc `kb status`, phân các section đang chờ ra các sub-agent chỉ đọc rồi gộp kết quả theo cùng những quy tắc. Các quy tắc (bản tóm tắt giữ ngôn ngữ của tài liệu và không bao giờ diễn đạt lại mã, con số, đơn vị hay tham chiếu chéo; bảng không bao giờ bị đụng tới) nằm trong skill. Section nào lỗi thì ở lại `pending`.

Để làm lại bản tóm tắt:

```
kb summarize hr-handbook --redo --section 4.12       # one section
kb summarize hr-handbook --redo --dry-run            # see the plan first
kb summarize --redo --all                            # every document
kb summarize hr-handbook --print-prompt --section 4.12
```

`--redo` bỏ qua các section đã được người review, trừ khi thêm `--include-reviewed --yes`.

3.5 Kiểm tra hợp lệ bằng `kb build`

```
kb build
```

```
kb build: OK
```

`kb build` thất bại khi còn dấu `TODO` hoặc tóm tắt rỗng, khi có bảng trong L2 không khớp bản gốc L3 (theo cả hai chiều, kể cả thứ tự; phép kiểm tra này không thể bỏ qua), khi L3 đổi sau khi tóm tắt đã viết, khi L2 đổi sau khi đã duyệt, và khi có phát hiện L3 chưa xác nhận. Quy tắc chất lượng (độ dài tóm tắt, độ trùng từ vựng với bản gốc, mã bịa ra) là cảnh báo; `--strict` biến chúng thành lỗi. Nó chỉ ghi số token vào manifest khi không báo lỗi nào.

```
kb build --allow-pending   # validate finished work while the rest is in progress
kb build --strict          # what kb approve requires
```

Khi lệch bảng, hãy chép nguyên bảng từ `.raw.md` (L3) của section đè lên bảng trong `.md` (L2). Đừng gõ lại hay định dạng lại.

3.6 Duyệt

Hãy kiểm tra các bản tóm tắt trước khi ký duyệt. Đọc các file `.md`, không phải `.raw.md`.

- Văn xuôi có khớp với hiểu biết của bạn về chủ đề không? Có điều quan trọng nào bị bỏ sót, hoặc có điều gì mà nguồn không nói?
- Mọi mã, định danh, con số và đơn vị có đúng y như trong nguồn không?
- Bảng: build đã so sánh, nhưng hãy liếc xem có lỗi trích xuất (một hàng hay cột mà bộ đọc PDF đọc sai) không; máy không bắt được điều đó.
- Các tóm tắt một dòng trong `_manifest.yaml` có nói đúng section nói về gì, đủ để tìm kiếm thấy nó không?

`kb approve` ký lên nội dung **đã commit**, nên hãy commit trước (`kb build` ghi lại số token trong manifest, vì vậy: build, rồi commit, rồi approve):

```
git add .kb/hr-handbook && git commit -m "kb: hr-handbook summaries"
kb approve hr-handbook                        # in your assistant: /kb-approve
kb approve hr-handbook --section 5.3          # only some sections (repeatable)
kb approve hr-handbook --by "Jane Smith <jane@acme.com>"
git add .kb/hr-handbook && git commit -m "kb: approve hr-handbook"
```

Nó từ chối cây `.kb/<id>` chưa commit và từ chối khi `kb build --strict` lỗi, chuyển các section `summarized` sang `reviewed`, và ghi `reviewed: {by, at, l2_sha256}` cho từng section (`--by` mặc định là danh tính git của bạn), nên một lần sửa sau đó vào bản tóm tắt đã duyệt sẽ bị phát hiện. `kb approve --all-changed --against <rev>` duyệt những gì đã đổi kể từ một revision. **Duyệt là hành động của con người và không bao giờ tự động**: trợ lý chỉ chạy nó khi bạn yêu cầu.

3.7 Publish

Xem trước, rồi publish:

```
kb diff hr-handbook --against HEAD   # added, removed and changed sections
kb publish                           # in your assistant: /kb-publish (diff, confirm, publish)
```

`/kb-publish` hiện diff, hỏi một lần đồng ý hay không, rồi chạy `kb publish`. Lệnh này in trước `kind: kb → hub menu: Document`, rồi:

Config của bạn	`kb publish` làm gì
Có `intake:`	Cần `.kb/` sạch và đã commit. Gắn tag `kb-publish/<ts>` vào commit, push tag, và CI của bạn publish (mục 4.2). Lệnh chờ tối đa 600 giây và in link PR.
Chỉ có `hub:`	Mirror `.kb/` vào `federation/<repo-id>/` trên hub và mở một PR ở đó. Cần `gh` mở được PR trên host của hub. Đường dẫn hub trên đĩa không có git remote thì được commit trực tiếp.

`--pr` và `--direct` ép một chế độ: `--direct` commit thẳng vào hub và bị từ chối trên hub đã được quản trị có git remote; không có cờ nào thì `kb publish` chỉ chọn direct với hub không có git remote, còn lại là một PR hoặc một lời từ chối rõ ràng. `kb publish` không có cờ `--intake`: nó đọc `intake:` từ config. `--hub` hoặc biến `STRATA_KB_HUB` ghi đè `hub:`.

Publish chỉ sao chép **artefact trong `.kb/`**: tài liệu, index, manifest và `assets/`. File cấu hình và dotfile ở lại, và mọi file bị giữ lại đều được nêu tên trong dòng `[warn] ... (allowlist)`. `_l3check.yaml` và `_l3fix.yaml` cũng bị giữ lại nhưng không có cảnh báo. Đây là chủ ý: một `config.yaml` mà ai đó đã để thông tin xác thực vào thì không bao giờ được tới hub.

`kb publish` và `kb ci-publish` cảnh báo có bao nhiêu section được gửi đi mà chưa review; `--require-reviewed` biến cảnh báo thành lỗi. Một phát hiện L3 chưa xác nhận luôn chặn. Nội dung tìm kiếm được khi PR được merge trên hub (hoặc ngay lập tức, nếu có auto-merge).

3.8 Đọc hub

```
kb query "parental leave eligibility"
kb query "parental leave" --tags hr --budget 600
kb get hr-handbook 4.12 --level l2     # or --level l3
kb tags                                # every tag published on the hub
```

Mỗi kết quả bắt đầu bằng `--- [<repo-id>:<doc-id> §<section> (<revision>)] match=<mode> ~<n>tk` (`keyword`, `semantic` hoặc `hybrid`) và trích dẫn đúng dạng đó. Hãy truy vấn bằng chính ngôn ngữ của tài liệu: kho không bao giờ dịch. `--budget` là trần token và chỉ mang tính tham khảo: kết quả đầu tiên luôn được trả về dù lớn đến đâu, vì section không bao giờ bị cắt. `--tags` cũng nhận id tài liệu; `--kind kb|ba|dev` lọc theo menu của hub. Cùng việc tìm kiếm này có sẵn cho trợ lý của bạn qua MCP (mục 2.4).

---

4. Các chế độ tự động

Child không có `kb auto`. Có bốn thứ chạy mà không cần bạn; không thứ nào duyệt hay xác nhận thay bạn.

Chế độ	Làm gì	Công tắc
Tự tóm tắt	`kb ingest` tóm tắt bằng CLI `claude` hoặc `copilot` mà nó tìm thấy	bật khi tìm thấy một CLI; `--no-summarize`, `--llm none`, hoặc `llm.runner: none` trong `.kb/index.yaml`
Sửa L3 bằng AI	một trợ lý sửa cấu trúc L3, máy kiểm tra (3.3)	`kb l3 fix on` / `off`
Publish CI qua OIDC	job GitHub Actions của chính bạn publish commit đã gắn tag	`intake:` trong `.kb/config.yaml`
Ingest bằng Docker	ingest chạy trong container, không cần Python cục bộ	`kb docker-setup`

`.kb/index.yaml llm:` gồm `runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers` và `timeout`. `--llm` trên `kb ingest` hoặc `kb summarize` ghi đè runner cho một lần chạy.

4.1 Điều gì luôn thuộc về con người

Duyệt không bao giờ tự động (3.6), AI không bao giờ xác nhận một phát hiện L3 hay một bản sửa của AI (3.2, 3.3), và việc bắt đầu publish là của bạn. Section được tự tóm tắt ở trạng thái `summarized`, chưa phải `reviewed`, cho đến khi bạn chạy `kb approve`.

4.2 Publish CI qua OIDC

`kb ci-publish` là đường CI: nó chạy trong job GitHub Actions của chính repo này, xác thực với dịch vụ intake của hub bằng GitHub Actions OIDC, và không cần bí mật nào. Workflow `.github/workflows/kb-publish.yml` đã dựng sẵn làm việc đó:

1. `kb publish` (khi có `intake:`) gắn tag `kb-publish/<ts>` vào `.kb/` đã commit và push tag.
2. Workflow, được kích hoạt bởi tag `kb-publish/*`, cài `strata-kb` (ghim theo phiên bản của hub, mục 2.4) và chạy `kb ci-publish` với `id-token: write`.
3. Intake mở PR trên hub (và merge nó, nếu Registry của hub bật auto-merge cho repo của bạn). `kb publish` poll để lấy link PR và hết thời gian chờ sau 600 giây.

Intake chỉ chấp nhận `refs/tags/kb-publish/*`. Trong repo child, tag đó là kích hoạt duy nhất: **merge vào nhánh mặc định không publish gì cả.** Mục registry của repo bạn phải để trống `workflow:`, vì bản ghim được so với một tham chiếu kết thúc bằng tên tag nên không bao giờ khớp. Người quản trị hub đăng ký `owner/repo` cùng `repo_id` của bạn; nếu `repo_id` khác, workflow vẫn thành công và PR vẫn được mở, nhưng `kb publish` hết thời gian chờ một PR mà nó không tìm thấy, và việc upload mất tính tăng dần (mỗi lần publish thành một lần upload đầy đủ).

`kb ci-publish` cũng nhận `--intake`/`STRATA_KB_INTAKE` và `--require-reviewed`; workflow chạy nó trần, nên hãy đặt `intake:` trong config.

4.3 Ingest bằng Docker

`docker-compose.yml` định nghĩa một service, `hub`, chạy ingest một lần trong image Strata: repo được mount tại `/data` và mô hình bố cục được cache trong một volume có tên.

```
docker compose run --rm hub kb ingest source/handbook.pdf --id hr-handbook --no-summarize
```

Container không có LLM CLI, đó là lý do `--no-summarize` được ghi rõ. Hãy tóm tắt sau đó trên máy của bạn (`kb summarize`) hoặc bằng `/kb-summarize`. `/kb-ingest` và `/kb-l3-fix` tự chọn container khi `docker info` chạy được và `docker-compose.yml` tồn tại, và gắn `docker compose run --rm hub` vào trước các lệnh `kb` của chúng.

---

5. Xử lý sự cố



6. Phụ lục

6.1 Khoá cấu hình và biến môi trường

Khoá hoặc biến	Ý nghĩa
`kind`, `hub`, `repo_id`, `intake`	xem 2.2. Không bao giờ đặt token vào `hub:` hay `intake:`
`l3_fix.enabled`	công tắc sửa L3 bằng AI, mặc định `false`; do `kb l3 fix on` / `off` đặt
`.kb/index.yaml` `llm:`	`runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers`, `timeout`
`STRATA_KB_HUB`	ghi đè `hub:`; mang URL có thông tin xác thực trong CI. Cũng được `kb publish --hub` và `kb doctor --hub` đọc
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	URL HTTP gốc và bearer token của hub: MCP và `kb doctor` qua MCP
`STRATA_KB_INTAKE`	giống `kb ci-publish --intake`
`STRATA_KB_HUB_CACHE`, `STRATA_KB_HUB_TTL`	bản clone hub cục bộ mà `kb query` đọc: thư mục của nó (mặc định `~/.strata-kb/hub/`) và thời gian nó được coi là còn mới (mặc định 900 giây)

`asset_store` chỉ dành cho hub: hình của child đi cùng bản publish.

Hai lưu ý. `kb mcp-setup --token` để lại token trong lịch sử shell: hãy dùng ô nhập ẩn hoặc `STRATA_KB_HTTP_TOKEN`. `kb publish` và `kb ci-publish` cảnh báo có bao nhiêu section được gửi đi mà chưa review; `--require-reviewed` biến cảnh báo đó thành lỗi.

6.2 Cái gì được commit, cái gì không

Đường dẫn	Chứa	Commit
`.kb/<id>/` (manifest, `.md`, `.raw.md`, `assets/`)	tài liệu	có
`.kb/<id>/_l3check.yaml`, `_l3fix.yaml`	trạng thái kiểm tra L3 và bản ghi sửa của AI	có, nhưng không bao giờ publish
`source/`, `sources/`	PDF nguồn	không, không bao giờ
`.kb-work/`	cache tạm của ingest, bản nháp sửa của AI, ảnh trang	không
`.env`	URL và token MCP	không (`kb mcp-setup` cho git bỏ qua nó)

6.3 Quy tắc skill sửa L3 bằng AI tuân theo

Skill `kb-l3-fix` chỉ chạy khi công tắc bật; nó ghi bản nháp và không bao giờ sửa `.kb/`; nó không bao giờ chạy `kb l3 confirm`; nó giữ nguyên văn bản của PDF (không thêm, không đổi lời, không dịch, không tóm tắt) và mọi bảng khớp từng byte; và nó thử lại một bản nháp bị từ chối tối đa hai lần rồi để section đó cho bạn. Chi tiết và thứ tự các phép kiểm tra nằm ngay trong skill.

6.4 Mã thoát

`0` ok, `1` lỗi hoặc từ chối (một lỗi `kb build`, `kb l3 report` khi còn phát hiện được liệt kê, `kb l3 fix apply` có bản nháp bị từ chối), `2` lỗi cách dùng, cache hub cũ trong `kb doctor`, hoặc citation cũ trong `kb resolve`.

6.5 Chạy lại `kb init`, Windows, và phía hub

`kb init` bỏ qua `.kb/config.yaml` và `.kb/index.yaml` khi chạy lại; `--force` thay thế chúng. File dựng khung tên `*.local.md` không bao giờ được làm mới, kể cả với `--force`. `kb doctor` chỉ làm mới các file của agent (`.claude/skills/`, `.claude/commands/`, `.github/prompts/`, `.cursor/commands/`).

`kb` ép UTF-8 cho các luồng của nó. Trên Windows, đường dẫn tài liệu dài cần `git config core.longpaths true`. `.gitattributes` giữ `.kb/**` không bị chuyển đổi kết thúc dòng (`-text`).

Tài liệu về hub nói về những gì người vận hành hub làm với bản publish của bạn: đăng ký repo, auto-merge và review PR.